1. How many patterns and problems?

Preparation level	Patterns mastered	Problems solved	Expected readiness
Foundation	8–10	60–100	Basic coding rounds
Strong	15–20	150–220	Many product-company interviews
Interview-ready target	20–25	200–300	Strong preparation for top product companies
Advanced	25–30+	300–450+	Broader coverage of difficult rounds

These are practical preparation targets, not guarantees of an offer. Company, role, seniority, and interview difficulty matter considerably.

2. The 25 DSA patterns I would master

Here is the roadmap I would follow, in roughly this order.

Aim for 8–15 problems per major pattern, fewer for straightforward topics and more for difficult ones. Some problems cover multiple patterns.

Phase 1 — Arrays and basic techniques

Patterns 1–6

1. Hashing and frequency counting
2. Two pointers
3. Sliding window
4. Prefix sums and difference arrays
5. Sorting, intervals and merging
6. Binary search and binary search on answer

Phase 2 — Core data structures

Patterns 7–12

7. Linked lists and fast/slow pointers
8. Stacks and monotonic stacks
9. Queues and monotonic queues
10. Heaps and top-K problems
11. Trees: DFS, BFS and recursion
12. Binary search trees and tree properties

Phase 3 — Graphs and search

Patterns 13–17

13. Graph DFS and BFS
14. Topological sorting
15. Union-Find / Disjoint Set Union
16. Shortest paths: Dijkstra, BFS variants
17. Backtracking and constraint search

Phase 4 — Dynamic programming

Patterns 18–22

18. Linear DP: take/skip, House Robber
19. Grid and two-dimensional DP
20. Knapsack and subset-sum DP
21. Subsequences and string DP, including LCS
22. Interval DP and partition DP

Phase 5 — Advanced patterns

Patterns 23–25

23. Greedy algorithms
24. Trie and prefix-based search
25. Bit manipulation and XOR

This is a practical grouping, not a universal taxonomy. Some interview problems combine three or four patterns, and advanced rounds may also require segment trees, Fenwick trees, or more specialized algorithms.

3. How many problems should you solve per pattern?

Don't assign the same number to every pattern. Use this as a starting allocation.

Pattern category	Approximate problems
Arrays, hashing, two pointers, sliding window	40–50
Binary search, prefix sums, intervals, greedy	30–40
Linked lists, stacks, queues, heaps	25–35
Trees and BSTs	25–35
Graphs and Union-Find	25–35
Backtracking	10–15
Dynamic programming	35–50
Tries, bit manipulation, other advanced topics	15–25
Total	205–285

Treat this as a flexible budget, not a checklist. Spend additional time on patterns you repeatedly fail to recognize.

4. What does it mean to truly master a pattern?

For each pattern, follow this sequence.

1. Understand the underlying idea. Know what property makes the technique applicable.

2. Learn the brute-force solution. Understand why it works and where its time complexity becomes expensive.
3. Study 2–3 representative problems. Trace the algorithm manually, including boundary cases.
4. Solve 5–10 problems independently. Start with easy problems, then medium and eventually selected hard problems.
5. Practice recognition. Attempt unfamiliar problems without looking at their tags or solution hints.
6. Re-solve after a delay. Revisit mistakes after 3–7 days and again after a few weeks.

For example, in binary search, you should recognize not just ordinary searching, but also first/last occurrence, lower and upper bounds, rotated arrays, and searching for the maximum or minimum feasible answer.

For DP, you should be able to derive the state, recurrence, base cases, evaluation order, and space optimization—not just reproduce a memorized solution.

5. A practical 16-week plan

Assuming you can dedicate roughly 1.5–2 hours per day, five or six days a week:

Weeks	Focus
1–3	Arrays, hashing, two pointers, sliding window, prefix sums
4–5	Binary search, sorting, intervals, stacks and queues
6–7	Linked lists, heaps, trees and BSTs
8–9	Graphs, topological sort, Union-Find, shortest paths
10–13	Recursion, backtracking and DP categories

14 Greedy, tries and bit manipulation

15–16 Mixed timed problems, revision and mock interviews